

Viva Questions & Answers

CredentialChain – Blockchain-Based Academic Credential System

Cecil Baraik · B.Sc. Information Technology (Batch 2025) · Amity University Jharkhand

Your 30-second introduction: "My project is CredentialChain, a blockchain-based academic credential system. Colleges issue certificates whose SHA-256 fingerprint is signed with ECDSA and stored on my own proof-of-work blockchain. Anyone can verify a certificate in seconds by scanning its QR code. If even one character is changed, the fingerprint no longer matches the blockchain, so fake certificates are caught immediately."

A. The project in simple words

Q1. What is your project?

CredentialChain is a blockchain-based academic credential system. A college issues digital certificates, and anyone (for example an employer) can check in seconds whether a certificate is real, fake or cancelled.

Q2. What problem does it solve?

Fake degree certificates. Today an employer has to email or phone the college to confirm a certificate, which is slow. In my system the employer just scans a QR code or enters the credential ID and gets the answer immediately.

Q3. Who are the users of the system?

Three: the **administrator** (registers colleges), the **college staff** (issue and revoke certificates) and the **verifier**, such as an employer, who needs no login. Students can also look up their certificates in the Student Wallet.

Q4. Explain the full flow in one minute.

The college fills the certificate form. The system makes a SHA-256 hash (fingerprint) of the data, signs that hash with the college's private key (ECDSA), and puts only the hash in a pending transaction. Mining puts it into a block with proof-of-work. To verify, the system re-calculates the hash from the data, compares it with the hash on the blockchain, checks the college's signature, checks it is not revoked, and validates the whole chain.

B. Blockchain concepts

Q5. What is a blockchain?

A list of blocks where every block stores the hash of the previous block. Because of this link, changing any old block changes its hash and breaks every block after it, so tampering is easy to detect.

Q6. What is inside one block in your project?

Index, timestamp, previous block hash, Merkle root of its transactions, nonce, difficulty, its own hash, mining time, and the list of transactions.

Q7. What is the genesis block?

Block #0, the first block. It has no transactions, and its previous hash is 64 zeros. Every chain starts from it.

Q8. What is proof-of-work and mining?

Mining means finding a number (the **nonce**) so that the block's hash starts with a fixed number of zeros. My difficulty is 4, so the hash must start with "0000". The computer tries thousands of nonces until one works. This makes rewriting history expensive, because an attacker would have to redo this work for the changed block and every block after it.

Q9. What is a nonce?

Just a counter the miner keeps changing. When the hash finally starts with the required zeros, that counter value is the nonce stored in the block. The dashboard shows how many attempts each block needed.

Q10. What is a Merkle root and why do you use it?

All transaction IDs in a block are hashed in pairs, again and again, until one hash is left: the Merkle root. It is stored in the block header, so it commits to every transaction. If any transaction changes, the Merkle root no longer matches.

Q11. What transaction types do you have?

Three: **REGISTER_ISSUER** puts a college's public key on the chain, **ISSUE** records a certificate's hash, and **REVOKE** cancels a certificate with a reason.

Q12. Is this a public blockchain like Bitcoin or Ethereum?

No. It is my own blockchain written in Java, running on one server node. It uses the same ideas (hash links, Merkle root, proof-of-work, digital signatures) but is permissioned: only registered colleges can add transactions. A future version could run on several nodes or on Ethereum / Hyperledger Fabric.

Q13. Why did you build your own blockchain instead of using Ethereum?

To understand and show every part myself: hashing, mining, Merkle tree and validation. It also runs offline with no gas fees, which is good for a college demo. The verification logic would stay the same if it were moved to a public chain.

C. Cryptography

Q14. What is SHA-256 and where do you use it?

A hash function that turns any data into a fixed 64-character fingerprint. Changing even one character gives a completely different hash, and you cannot get the data back from the hash. I use it for certificate fingerprints, transaction IDs, block hashes and the Merkle root.

Q15. What is a digital signature? Which algorithm did you use?

ECDSA on the P-256 curve (SHA256withECDSA). The college signs with its **private key**, which only it holds, and anyone can check the signature with its **public key**. It proves who issued the certificate and that the data was not changed.

Q16. Where are the colleges' public keys stored, and why there?

On the blockchain, through a REGISTER_ISSUER transaction. Even if someone hacks the database and replaces a key, verification still uses the key on the chain, so the fake key does not work.

Q17. Why do you store only the hash on the blockchain and not the full certificate?

Privacy. A blockchain cannot be edited or deleted, so personal data should not go there. The full record stays in the database (off-chain) and only the fingerprint goes on-chain. That is enough to prove the data is unchanged.

D. Verification and attacks

Q18. What checks happen when someone verifies a certificate?

Six: record exists, anchored on the blockchain, data matches the on-chain hash, issuer signature is valid, not revoked, and the whole ledger is intact. All must pass for a VALID result.

Q19. What happens if a student edits the grade in the downloaded file?

The recalculated hash changes, so it no longer matches the hash on the blockchain, and the signature no longer fits the data. The result is VERIFICATION FAILED. I showed this live by changing 9.0 CGPA to 10.0.

Q20. What if a corrupt staff member changes the grade directly in the database?

Same result: the database record's hash no longer matches the on-chain hash, so verification fails. I demonstrate this with the Attack lab's "Forge grade" button.

Q21. What if a hacker edits a block that is already on the chain?

"Validate entire chain" recomputes every transaction ID, Merkle root, block hash, link and signature. The edited transaction's ID and signature no longer match, so the block is reported as tampered. The Attack lab's "Tamper block" button shows this.

Q22. How does revocation work if the blockchain cannot be changed?

We never delete. The college adds a new REVOKE transaction with a reason. Verification then shows "Authentic, but revoked", and the full history stays auditable.

E. Security and login

Q23. How are passwords stored?

Never as plain text. I use PBKDF2-HMAC-SHA256 with a random salt per user and 210,000 iterations, which is built into Java and recommended by OWASP. Comparison is constant-time.

Q24. How does login work after signing in?

The server gives a JWT token signed with HMAC-SHA256. It contains the user ID, role and expiry (8 hours). If anyone edits the token, for example changing their role to ADMIN, the signature check fails and the request is rejected.

Q25. What roles exist and what can each do?

ADMIN registers colleges and uses the Attack lab. ISSUER (college staff) issues and revokes only their own college's certificates. Verifying is public and needs no login.

Q26. How do you stop password guessing?

After 5 wrong attempts the account is locked for 5 minutes. The error message is the same for a wrong email and a wrong password, so attackers cannot find out which emails exist.

F. Technology and implementation

Q27. Which technologies did you use?

Backend: Java 17+, Spring Boot 3, Spring Data JPA. Database: H2 by default and MySQL. Frontend: React 18 with Vite. Extras: jsPDF and QRCode for PDF certificates, and JUnit + MockMvc for tests.

Q28. Why Spring Boot and React?

Spring Boot gives a ready REST API, database access and validation with very little setup. React makes the interface interactive. They talk through a REST API, so each part can be changed independently.

Q29. How is the database designed?

Five tables: **blocks** (each block saved as JSON), **pending_transactions** (the mempool), **credentials** (full certificate data, off-chain), **institutions** (colleges and their keys) and **user_accounts** (logins with hashed passwords). For MySQL the app uses its own user that can only access the credchain database.

Q30. How did you test the project?

With JUnit unit tests for hashing, signatures, mining and the Merkle tree, and a full integration test that goes through login, issue, mine, verify, a forged file, database and block tampering, restore and revoke. I also tested every flow manually in the browser.

G. Limitations and future scope

Q31. What are the limitations of your project?

It runs on a single node, so it is not decentralised yet. College private keys are kept on the server for the demo; in a real system each college would keep its own key in a wallet or hardware security module. QR codes work on other devices only when the app is reachable on the network.

Q32. How would you improve it in future?

Multiple nodes with a consensus algorithm, or moving the smart logic to Ethereum or Hyperledger Fabric. Also W3C Decentralized Identifiers (DIDs), two-factor login for staff, email delivery of certificates to students, and storing documents on IPFS.

H. Tips for the viva

- Start with the problem (fake certificates), then show the solution live. Examiners remember demos.
- Demo order: Dashboard → Issue → Mine block → Verify (valid) → Upload edited file (failed) → Revoke → Attack lab.
- If you don't know an answer, say what you do know and how you would find out. Don't guess.
- Keep the app running and signed out before the viva, so you can show that verification needs no login.

Key terms in one line

Term	Meaning in one line
Hash (SHA-256)	A 64-character fingerprint of data; any change gives a different hash
Block	A page of the ledger holding transactions and the previous block's hash
Nonce	The number miners change until the hash starts with enough zeros
Proof-of-work	The computing effort that makes rewriting blocks expensive
Merkle root	One hash that represents all transactions in a block
ECDSA signature	Proof made with a private key, checked with the public key
Revocation	A new REVOKE transaction that cancels a certificate without deleting history
JWT	A signed login token carrying the user's ID, role and expiry
PBKDF2	A slow, salted password-hashing method so stolen hashes are hard to crack